

LC 206 Reverse Linked List

Approach:

- Maintain Create two pointers that stores the head and next node from head in the linked list
- Create a temporary pointer and point it to null, this will be useful when needed to store the node elements
- If head and next node from head is null return head
- If head not null, store head in temp, move n forward, reverse the link, move head pointer forward,
- Terminate the original head and return new head

LC 876 Middle of a linkedlist

Approach:

- Create Initialize two pointers slow and fast , which traverses by one node and two nodes respectively, map them to head
- When fast and next of fast not equal to 0, move slow by one node and fast by two nodes,
- What happens is when fast reaches 0 or fast->next reaches zero then slow is at the middle of the list
- Return slow

LC 21 Merge to sorted lists

Approach:

- Need to compare both nodes in the list and then store the smaller one
- Create a dumminode and its tail pointer to store the smaller nodes
- When list 1 node is smaller than list 2 node, attach the list1 node and move forward, else do the same for list 2 node
- Move temp to next node
- Attach remaining nodes
- Return head of merged list